Testování softwaru

Přednáška 4:
Techniky pro návrh testovacích scénářů:
životní cyklus datových objektů

Miroslav Bureš 2016

Úvod

- **Odhalí vykonávání akcí za sebou v aplikaci všechny možné stavy, související s konzistentním zpracováním dat?**
  - Aplikace neztrácí nebo nepoškozuje části dat
  - To co uložím, poté správně přečtu
  - Funkcionalita nezhavaruje vlivem nekonzistentních dat

Zkuste najít příklad, kdy procesní testy neodhalí nekonzistenci dat

Úvod

- **Odhalí vykonávání akcí za sebou a aplikaci všechny možné stavy, související s konzistentním zpracováním dat?**
  - Aplikace neztrácí nebo nepoškozuje části dat
  - To co uložím, poté správně přečtu
  - Funkcionalita nezhavaruje vlivem nekonzistentních dat

- **Příklad:**
  - S konkrétní datovou entitu **klient** pracují procesy A, B, C
  - Procesy A, B, C otestuji procesním testem s hloubkou pokrytí 2 (viz předchozí přednáška)
  - Testy procesů projdou bez nalezení chyby
  - Stejně může uniknout chyba:
    - Akce X v procesu A nějakým způsobem poškodila část datové entity
    - Při vyvolání dalších akcí z procesu A se na to nepřišlo – tyto akce nepracovaly s poškozenou částí dat
    - Nicméně s touto částí dat pracovala akce Y v procesu C, která díky tomu zhavarovala

Test životního cyklu dat

- **Data uložená v testovaném systému mají svůj životní cyklus**

  Typicky sekvence:
  - Založení datové entity **C (create)**
  - Čtení dat **R (read)**
  - Aktualizace dat **U (update)**
  - Smazání datové entity **D (delete)**

  Read a Update se typicky může různě víckrát opakovat

- Obecně:
  - Kompletní životní cyklus datové entity se zpravidla neodehrává v jednom procesu
  - Mezi procesy a datovými entitami je obecně M:N vazba
    - Proces může pracovat s více datovými entitami
    - Konkrétní datová entita může být zpracovávána více procesy, které vyvolávají operace C,R,U,D
  - Pokrytí procesů tedy nezajišťuje pokrytí celého životního cyklu dat

CRUD

- **Jaké jsou možné chyby?**
  - Některá z operací C, R, U, D není vůbec implementovaná
  - Některá z operací C, R, U, D neprobíhá správně
  - Pokud se operace C, R, U, D vykonají mimo očekávaný průběh, naruší se konzistence dat
  - Operace D není ve sktečnosti delete, ale jen archivace. Ale některé z funkcí tesotvaného systému pracují s archivovanou entitou stále jako s aktivní...
  - …

CRUD matice

- Na projektu může být součástí analytické a návrhové dokumentace

- Pokud není k dispozici, vytvoříme ji. Vstupy:
  - Seznam funkcí aplikace
  - Seznam datových entit aplikace
  - Informace, jaké akce C, R, U, D jsou vykonávané jakou funkcí nad jakými datovými entitami

- Příklad CRUD matice:

	Klient	**Objednávka**	**Sestava**	**…**
Založení klienta	**C**	-	-	
Editace klientských údajů	**R, U**	**R**	-	
Nákup košíku	**R**	**C**	-	
Vytvoření účetní sestavy	**R**	**R**	**C**	
Zrušení objednávky	-	**D**	-	
…				

Testy pomocí CRUD matice

Test kompletnosti

- Statické testování
- **Kontrola CRUD matice**, jestli má každá datová entita pokryty všechny 4 operace C, R, U, D
- Pokud nemá, nemusí to nutně znamenat chybu. Znamená to ale situaci, kterou je potřeba prověřit.

Test datové konzistence

- Dynamické testování
- Kontrola, jestli všechny funkce aplikace zachovávají použité data entity v konzistentním stavu
- **Nestane se, že nějaká funkce pokazí datovou entitu tak, na ní pak jiná funkce může zhavarovat (nebo nebude fungovat správně)?**

Test datové konzistence

Základní pravidla:

- Sestavujeme scénář z C, R, U, D operací vyvolaných funkcemi aplikace

- Scénář začíná C, pak následuje kombinace R a U, scénář končí D
- Po každém C, U, D vyvolejte R
  - Jednou pro nejdůležitější funkci, která zobrazuje datovou entitu
    - **základní pokrytí**
  - Vícekrát pro různé funkce, které zobrazují datovou entitu
    - **kompletní pokrytí R**

- Pokud kvůli kapacitě není možné pokrýt všechny operace C, R,U, D, které dělají všechny funkce v aplikaci, řiďte se prioritou funkcí.

- Pozor při vyvolávání funkcí, kterými v programu „doputujeme“ k požadované operaci C, R, U, D. Nezmění náhodou „bezděčně“ jiná data?

Jednoduchý příklad

	Klient	Objednávka	Sestava
Vytvoření objednávky	**R**	**C**	-
Aktualizace objednávky	-	**U**	-
Zobrazení objednávky	-	**R**	-
Seznam objednávek	-	**R**	**C**

Funkce ***aktualizace objednávky*** způsobí chybu v metadatech, která se projeví ve funkci ***seznam objednávek***, ale ne ve funkci ***zobrazení objednávky***

Základní pokrytí:

- Vytvoření objednávky
- Zobrazení objednávky
- Aktualizace objednávky **(chyba poškodí data)**
- Zobrazení objednávky

(chyba unikne)

Kompletní pokrytí R:

- Vytvoření objednávky
- Zobrazení objednávky
- Seznam objednávek
- Aktualizace objednávky **(chyba poškodí data)**
- Zobrazení objednávky
- Seznam objednávek **(chyba nalezena)**

CRUD - vázané podmínky

- S konkrétní datovou entitou A jde provést nějaká z operací C, R, U, D, pouze pokud je splněná daná podmínka

- Příklady:
  - Entitu zákazník lze smazat, pokud má zaplacené všechny objednávky
  - Provozovnu lze smazat, pokud do ní nejsou přiřazení žádní zaměstnanci

- Pozor, taková podmínka nemusí být jen na úrovni integrity v DB schématu!

- Testujeme pozitivní i negativní variantu, tj.
  - Podmínka je splněná a akci lze korektně provést
  - Podmínka není splněná a systém má konzistentně ošetřit chybu

Pohledy na aplikaci

Předchozí pohledy na aplikaci:

- **Pokrytí procesů**
- **Pokrytí datových cyklů**
- **Stačí to? ;)**

Zkuste najít příklad, kdy to nestačí

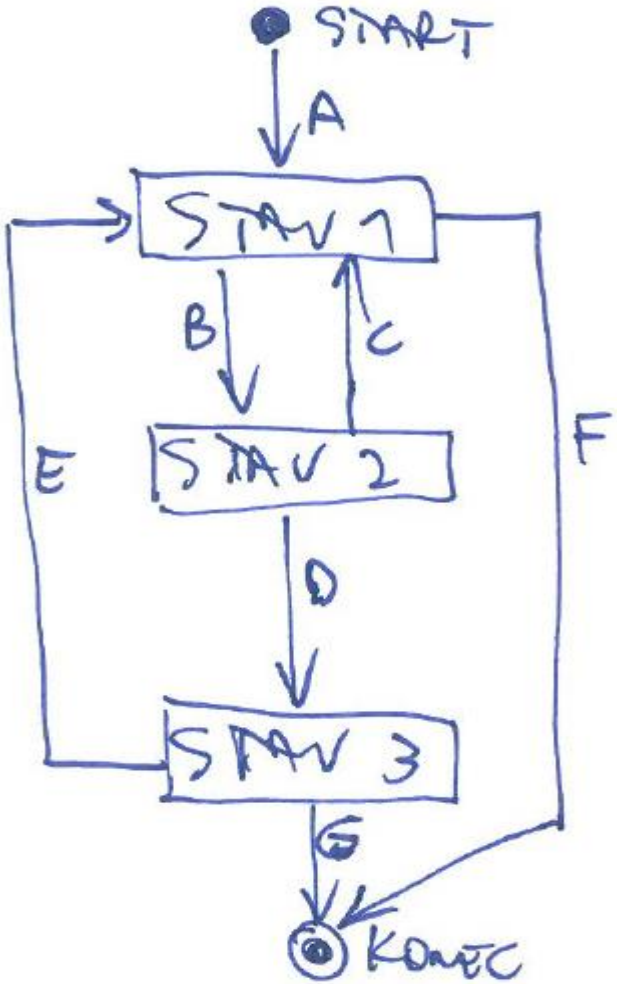
Test přechodu stavů

- Pokud je část aplikace (nebo specifikace) řešena stavovým automatem, stavy a přechody automatu:
  - Nemusí být 1:1 s procesním diagramem
  - Přechody stavů jsou zpravidla operace U z CRUD pohledu
  - Ale CRUD test není 1:1 s přechody automatu

- V těchto situacích má smysl použít další typ testů – **test přechodu stavů** (State transition test)

Test přechodu stavů

Příklad stavového automatu



Notace se mohou lišit:

- viz UML zápis
- Označení počátečního a koncového stavu
- Co když není označeno?

Test přechodu stavů

Zápis stavového automatu pomocí tabulky

	A	B	C	D	E	F	G
Start	Stav 1						
Stav 1		Stav 2				Konec	
Stav 2			Stav 1	Stav 3			
Stav 3					Stav 1		Konec
Konec							

Minimální kontrola konzistence, pokud stav. automat vytváříme ze specifikace:

- Všechny akce pokryté
- Všechny stavy pokryté
- Definovaný počáteční a koncový stav
- Do stavu se lze dostat
- Kromě koncového stavu musí ze všech jít přechod ven

Test přechodu stavů

Testovací pokrytí:

N-switch coverage, N=0, 1, 2, …

Testy pokrývají všechny platné sekvence N+1 navazujících transakcí

Pokrytí	Popis	Kdy použít (typicky)
0-switch coverage	Každý z přechodů je vykonán jednou	Testy málo prioritních akcí, smoke testy
1-switch coverage	Testy pokrývají všechny platné sekvence 2 navazujících transakcí	Střední intenzita testů
N-switch coverage, N>=2	Viz definice, testy pokrývají všechny platné sekvence N+1 navazujících transakcí	Vysoká intenzita testů

Jaký je počet testovacích situací pro 0-switch coverage?

0-switch coverage

Testovací případy

	Test						
	1	**2**	**3**	**4**	**5**	**6**	**7**
Počáteční stav	Start	Stav 1	Stav 1	Stav 2	Stav 2	Stav 3	Stav 3
Akce	A	B	F	C	D	G	E
Koncový stav	Stav 1	Stav 2	Konec	Stav 1	Stav 3	Konec	Stav 1

1-switch coverage

Testovací případy

	Test											
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Počáteční stav	Start	Start	Stav 1	Stav 1	Stav 1	Stav 2	Stav 2	Stav 2	Stav 2	Stav 3	Stav 3	Stav 3
Akce	A	A	B	B	F	C	C	D	D	G	E	E
Další stav	Stav 1	Stav 1	Stav 2	Stav 2	Konec	Stav 1	Stav 1	Stav 3	Stav 3	Konec	Stav 1	Stav 1
Akce	B	F	C	D		B	F	G	E		B	F
Koncový stav	Stav 2	Konec	Stav 1	Stav 3		Stav 2	Konec	Konec	Stav 1		Stav 2	Konec

Jak se dá mechanicky nakombinovat z 0-switch coverage?

Má smysl spojit testy do delší sekvence kroků?

Error guessing

Pomocí předchozích test design technik jsme pokryli aplikaci z různých pohledů a principů funkčnosti.

Jak ještě doplnit testovací pokrytí, tj. jak identifikovat místa, kde ještě mohou být další chyby?

- Error guessing
- Neformální technika, založená na zkušenosti testera, ale nemusí být podmínkou (možné zapojit budoucí uživatele, administrátory, …)
- **Odhadneme „kde se v systému mohou vyskytnout chyby“**, vzhledem k:
  - Historii vývoje a chybovosti
  - Kvalitě specifikace a nedostatečně popsaným komponentám
  - Stabilitě systémů třetích stran a rozhraní, způsobu integrace
  - Subjektivní zkušenosti testerů
  - Defect clustering
  - ….
- Pokrytí formálně těžko vykazovatelné
- Stejně jako v exploratory testing:
  - Můžeme zaznamenávat provedené testy – silně doporučeno
  - Podporuje kreativní testování – motivace testovacího týmu

Error guessing

<u>Situace vhodné pro error guessing</u> *(podle TMAP Next, doplněno)*

- Výjimečné situace v běhu systému
- Ošetření chyb
  - Úmyslné zavlečení chyby
  - Neočekávané přerušení procesu
- Technicky nepovolené hodnoty, hodnoty mimo rozsah (pokud již není pokryto v předchozích testech)
- Specifické kombinace dat
  - Které nejsou pokryty v předchozích kombinacích a adresují mezní podmínky
  - Sekvence transakcí mimo průběhy definované formálními technikami, časté opakování konkrétních kroků (pokud neřeší zátěžové testy), se zapojením ošetření chybových stavů
- Požadavky na zdroje systému (pokud neřeší zátěžové testy)
- Snaha zasílat systému požadavky v průběhu zpracování (pokud neřeší zátěžové testy)
- Odhad situací, které může vyvolat „naivní“ uživatel, případně jeho zapojení

Error guessing

Situace vhodné pro error guessing *(podle TMAP Next)*

- Komplexní části systému
- Často měněné části systému
- Části systému, které historicky obsahovaly hodně chyb

General way of thinking suitable for error guessing

- "crash test" – we are trying to make the system fail

Error guessing

Srovnání s Exploratory testing *(podle TMAP Next)*

Error guessing	Exploratory testing
Nepoužívá základní test-design techniky	V konkrétní situaci při procházení aplikace ad-hoc používám základní test-design techniky
Vhodné pro testery, uživatele, administrátory Zkušenost testerů je výhoda	Vhodné pro zkušené testery, kteří znají základní techniky
Návrh scénářů během přípravy nebo rovnou během testování, ale není podmínkou	Návrh scénářů během testování
Zaměření na výjimky, komplikované situace a odhad chyb mimo stávající pokrytí	**Zaměření na pokrytí aplikace testy**
Nemusí být systematická v procházení apliakce	Systematická v procházení aplikace

Operační profily

Jakým způsobem se dá pro návrh testů zjistit, jakým způsobem bude aplikace nejčastěji používána?

Operační profil

- Popisuje, jak je systém užíván konkrétním typem uživatele
- Neformálně: Odhad nebo záznam aktivit nebo transakcí uživatelů v aplikaci za konkrétní časový úsek

Rozšiřujeme stávající aplikaci:

- Získáme informace o jejím předchozím provozu

V jaké situaci to může být naopak zavádějící?

Vytváříme novou aplikaci:

- Operační profil podobné aplikace nebo procesu

Operační profily

Jak získat data pro operační profil

- Logy aplikace:
  - Samotné aplikace, pokud vytváří logy
  - Logy webového / aplikačního serveru
  - Někdy jde částečně odvodit z logů DB serveru
  - Možnost nainstalovat monitoring na server / uživatelské stanice

- Analýza specifikace
- Odhady od budoucích uživatelů (specialista na business doménu)
- Zkušenost s podobnou aplikací nebo procesem

Napadl by vás ještě nějaký způsob?

Operační profily

Vytvoření operačního profilu

	Akce v aplikaci				
	Akce 1	**Akce 2**	**Akce 3**	**Akce 4**	**…**
Stav aplikace 1	10%	15%	60%	10%	
Stav aplikace 2	1%	20%	10%	50%	
Stav aplikace 3	0%	5%	0%	80%	
…					

- V buňkách tabulky jsou pravděpodobnosti vyvolání akce v daném stavu aplikace, uvedené v %
- Součet pravděpodobností v řádku je 100%

Generování testů

- Pro každý stav rozložení akcí na intervalu 0-100 (akce 1: 0-9, akce 2: 10-25, ...)
- Generování náhodného čísla 0-100, výsledný set bude respektovat rozložení
- Potřeba respektovat dostupnost akcí v jednotlivých stavech a možné přechody

YOU ELIMINATED THE BUDGET FOR AUTOMATED TEST SOFTWARE. HOW ARE WE GOING TO TEST OUR NEW CODE?
scottadams@aol.com
www.dilbert.com
GO WRITE SOME AUTOMATED TEST SOFTWARE, YOU BIG BABY. I ALREADY PAY YOU, SO IT'S FREE.
7-28-07 ©2007 Scott Adams, Inc./Dist. by UFS, Inc.
TODAY I LEARNED YOU CAN END ANY CONVER-SATION BY CALLING THE OTHER PERSON A BIG BABY.
WAA-WAA! DO YOU WANT YOUR BOTTLE?

Děkuji za pozornost

buresm3@fel.cvut.cz

Zdroje

TMAP Next:
http://www.tmap.net/en/tmap-next
T. Koomen, L. van der Aalst, B.Broekman, M. Vroon: TMap Next, UTN Publishers, 2010

Test přechodu stavů:
E. van Veenendaal, J.Seubers: Black Box Techniques IN: E. van Veenendaal, The Testing Practitioner, UTN Publishers, 2002, pp. 227-260

ISTQB Certified tester Foundation level syllabus, release 2011
http://www.istqb.org/downloads/viewcategory/16.html

ISTQB Glossary of testing terms, version 2.2
http://www.istqb.org/downloads/viewcategory/20.html